

MASTER 2 — DEVOPS, CLOUD & INFRASTRUCTURE

Sécuriser la chaîne d'approvisionnement logicielle

SBOM · Signature (cosign/Sigstore) · Attestations SLSA · Admission Kyverno

Membres du groupe

Khalid OUATIK

Omar Mohammed HASSAS

Loic STEVE

Hadama TOURE

Dépôt GitHub du POC :

<https://github.com/KhalidOuatik/projet-technique>

08 juillet 2026

« On ne fait pas confiance, on vérifie. »

Rapport — Chaîne d'approvisionnement logicielle sécurisée

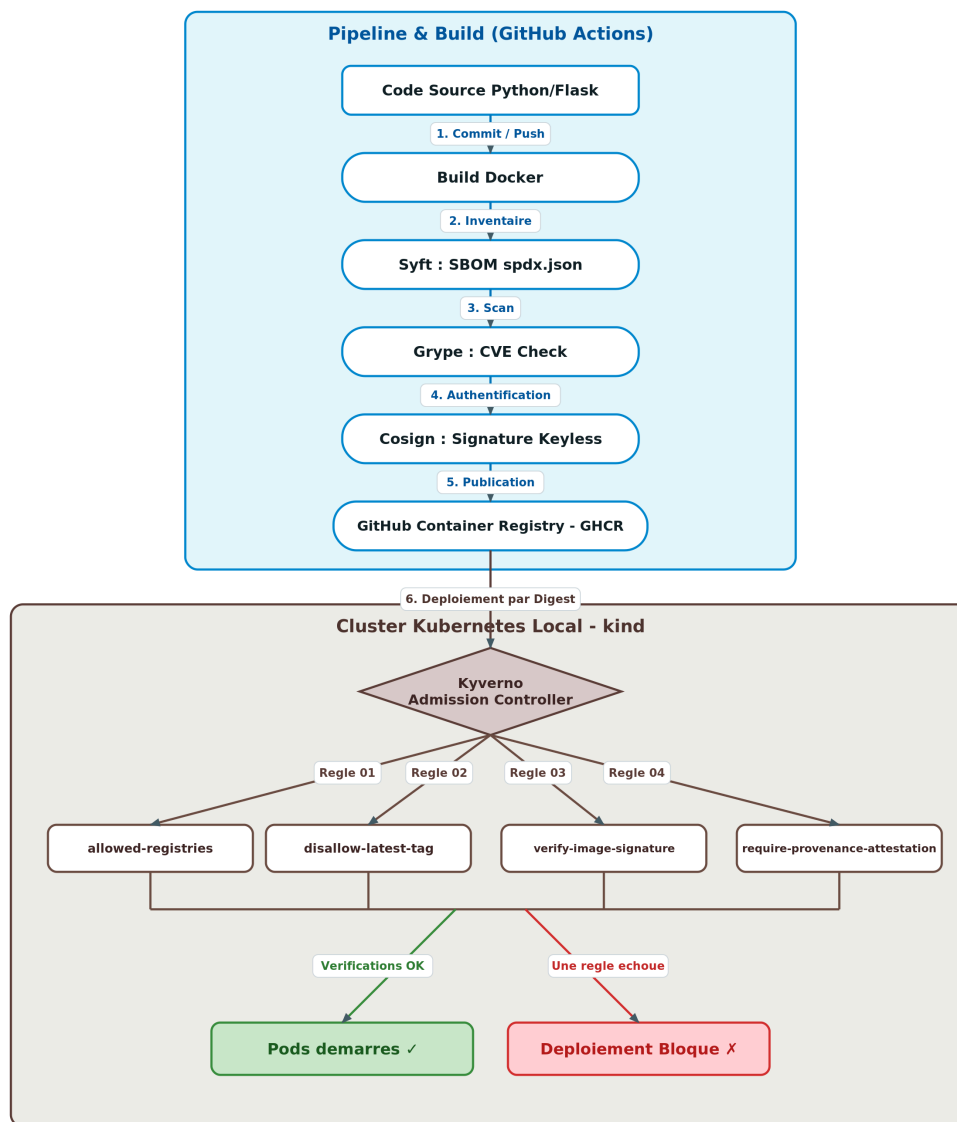
- **Groupe** : Khalid OUATIK · Omar Mohammed HASSAS · Loic STEVE · Hadama TOURE
- **Fork** : <https://github.com/KhalidOuatik/projet-technique>
- **Voie** : ☒ Local (kind) + CI GitHub Actions (Lab 5 bonus)
- **Date** : 08 Juillet 2026
- **Threat model associé** : inclus en seconde partie de ce document

1. Contexte & objectif

Ce projet vise à sécuriser la chaîne d'approvisionnement logicielle (Software Supply Chain) d'une API Flask. L'objectif est de répondre à la question : **comment prouver que l'image qui tourne en production est bien celle que nous avons construite — et pas une version piégée ?**

Un `docker pull` ne vérifie rien, et « le scan était vert » ne prouve pas que l'image *déployée* est celle qui a été scannée. La réponse mise en œuvre : SBOM, signature cosign liée au **digest**, attestations de provenance (SLSA) et contrôle à l'**admission** (Kyverno, policy-as-code) — le passage de « on scanne » à « on vérifie et on bloque ».

2. Architecture de la chaîne



Deux voies de build coexistent :

- **Voie locale** (démonstration attaque/défense) : build Docker local, signature par **clé** cosign, cluster kind.
- **Voie CI** (Lab 5) : GitHub Actions build, scanne (gate bloquant), signe en **keyless** (OIDC, journalisé dans Rekor) et attache les deux attestations — sans aucune clé stockée.

3. Mise en œuvre

SBOM (Syft)

Généré aux formats SPDX et CycloneDX :

```
syft ghcr.io/khalidouatik/scs-demo-app:0.1.0 -o spdx-json > sbom.spdx.json
syft ghcr.io/khalidouatik/scs-demo-app:0.1.0 -o cyclonedx-json > sbom.cdx.json
```

- `sbom.spdx.json` (2.3 Mo) — ~140 composants inventoriés (paquets Debian de l'image de base + dépendances Python)
- `sbom.cdx.json` (0.98 Mo)

Le SBOM est aussi **attesté** (signé et attaché à l'image, voir plus bas) : ce n'est pas un fichier posé à côté, c'est une affirmation vérifiable liée au digest.

Scan de vulnérabilités (Grype) — le gate qui casse

Configuration `.grype.yaml` à la racine (utilisée en local **et** en CI) :

```
only-fixed: true
fail-on-severity: critical
```

Choix assumé : on ne bloque que sur les CVE **critiques pour lesquelles un correctif existe** (`only-fixed`). Bloquer sur des CVE sans correctif rendrait le gate inactionnable (l'équipe ne peut rien corriger) ; ces CVE restent néanmoins tracées via le SBOM.

Preuve du gate — sur une version volontairement vulnérable (`Flask==2.0.1`, tag `:vuln`) :

```
grype ghcr.io/khalidouatik/scs-demo-app:vuln --only-fixed --fail-on critical
# [0002] ERROR discovered vulnerabilities at or above the severity threshold
# Code de sortie : 2 → le pipeline s'interrompt ici
```

En CI, le même gate est actif dans `.github/workflows/supply-chain.yml` (`fail-build: true`, `severity-cutoff: critical`, `only-fixed: true`) : un build contenant une CVE critique corrigeable **ne produit pas d'image**. Sur l'image saine actuelle, le scan passe (exit 0 — aucune CVE critique corrigeable).

Signature et attestations (cosign)

Génération des clés :

- Clé privée : `cosign.key` — **jamais committée** (`.gitignore`, vérifié sur tout l'historique)
- Clé publique : `cosign.pub` — versionnée et collée dans les politiques Kyverno

```
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAELZvgkuGC00sdPnhVo1bCz5nccDW7
4z0MAggCTrSeoRtY10XxY9Z37tqbfno00IBGsDvoKYhpE3xYK1nC716fuQ==
-----END PUBLIC KEY-----
```

Signature et attestations — toujours **par digest**, jamais par tag :

```
IMG=ghcr.io/khalidouatik/scs-demo-
app@sha256:edf18d5cbdda8c97187310ba95a99dc33876293f7f18cce8c8a473000753fea1
cosign sign --key cosign.key $IMG
cosign attest --key cosign.key --predicate sbom.spdx.json --type spdxjson $IMG
cosign attest --key cosign.key --predicate provenance.json --type slsaprovenance $IMG
```

Incident rencontré (et résolu) — incompatibilité de format cosign v3 / Kyverno 1.12 : cosign v3 stocke par défaut signatures et attestations via l'API *OCI referrers* (nouveau format bundle), alors que Kyverno 1.12 recherche l'ancien format par tag (`sha256-<digest>.sig / .att`). Résultat : `cosign verify` réussissait en local, mais Kyverno répondait `no signatures found` et refusait notre propre image légitime. **Résolution :** re-signature avec cosign **v2.5.3** (format legacy par tag), immédiatement comprise par Kyverno. Leçon retenue : dans une chaîne de confiance, le *format de preuve* fait partie du contrat entre outils — à épingler (versions) comme le reste.

Admission (Kyverno 1.12.6)

Les 4 politiques sont déployées en mode **Enforce** (vérifié : `kubectl get clusterpolicy`, capture `livrables/captures/policies-enforce.txt`) :

Politique	Rôle	Action
<code>allowed-registries</code>	Seul <code>ghcr.io/khalidouatik/</code> est autorisé	Enforce
<code>disallow-latest-tag</code>	Tag : <code>latest</code> ou absence de tag interdits	Enforce
<code>verify-image-signature</code>	Signature cosign valide avec notre clé exigée	Enforce
<code>require-provenance-attestation</code>	Attestation <code>slsaprovenance</code> signée exigée	Enforce

Kyverno s'authentifie auprès du registry GHCR (privé) via le secret `kyverno/kyverno-registry-credentials` pour récupérer signatures et attestations.

4. Comment vérifier (commandes exactes)

Toute affirmation de ce rapport est vérifiable :

```
# 1. La signature est valide et liée au digest
cosign verify --key cosign.pub \
  ghcr.io/khalidouatik/scs-demo-
app@sha256:edf18d5cbdda8c97187310ba95a99dc33876293f7f18cce8c8a473000753fea1

# 2. L'attestation SBOM est attachée et signée
cosign verify-attestation --key cosign.pub --type spdxjson \
  ghcr.io/khalidouatik/scs-demo-app@sha256:edf18d5c... | jq '.payloadType'

# 3. L'attestation de provenance est attachée et signée
cosign verify-attestation --key cosign.pub --type slsaprovenance \
  ghcr.io/khalidouatik/scs-demo-app@sha256:edf18d5c... | jq -r '.payload' | base64 -d | jq
'.predicate'

# 4. L'arbre complet des artefacts attachés
cosign tree ghcr.io/khalidouatik/scs-demo-app@sha256:edf18d5c...

# 5. Les politiques sont bien en Enforce
kubectl get clusterpolicy
```

Sortie de `cosign verify` (capture complète : `livrables/captures/cosign-verify.txt`) :

```
Verification for ghcr.io/khalidouatik/scs-demo-app@sha256:edf18d5c... --
The following checks were performed on each of these signatures:
- The cosign claims were validated
- Existence of the claims in the transparency log was verified offline
- The signatures were verified against the specified public key
```

5. Démonstration attaque / défense

Exécution réelle de `./demo.sh` le 08/07/2026, politiques en `Enforce` (**sortie terminal complète** : `livrables/captures/demo-output.txt`). L'image « modifiée après signature » est préparée par `./attack-tamper.sh` : rebuild de l'app avec une route `/backdoor` injectée, poussée sous le tag d'apparence légitime `1.0.0-tampered`.

#	Scénario	Résultat	Contrôle déclenché	Message Kyverno (réel)
0	Image légitime (par digest)	✅ acceptée	—	<code>deployment.apps/scs-demo-app created</code> — 2 pods Running
1	Image non signée	❌ refusée	<code>verify-image-signature + require-provenance-attestation</code>	<code>no signatures found / no matching attestations</code>
2	Tag mutable : <code>:latest</code>	❌ refusée	<code>signature</code> (première à bloquer) et <code>disallow-latest-tag</code>	<code>no signatures found</code> — et même signée, le tag <code>:latest</code> resterait interdit par la politique 02
3	Registry non autorisé (nginx:alpine)	❌ refusée	<code>allowed-registries</code>	Image refusée : seules les images de <code>ghcr.io/khalidouatik/</code> sont autorisées
4	Image modifiée après signature (backdoor)	❌ refusée	<code>verify-image-signature</code>	<code>no signatures found</code> — le contenu a changé → le digest a changé → aucune signature ne correspond

Le point clé (scénario 4) : l'attaquant n'a *pas besoin* d'être détecté par un scanner. Un seul octet modifié suffit à changer le digest SHA-256 ; la signature — liée au digest de l'image légitime — ne correspond plus, et le cluster refuse **avant même** de créer le Pod. C'est la garantie d'intégrité de bout en bout.

Extrait de la sortie réelle de `./demo.sh` (intégralité dans `livrables/captures/demo-output.txt`) :

```
[Cas Nominal] 1. Déploiement de l'image signée & attestée...
deployment.apps/scs-demo-app created
--> SUCCÈS : Le cluster a accepté le déploiement de l'image légitime. ✅

[Attaque 1] Tentative de déploiement d'une image NON SIGNÉE...
resource Pod/app/pirate-unsigned was blocked due to the following policies
verify-image-signature:
  verifier-signature-cosign: '... .attestors[0].entries[0].keys: no signatures found'
--> DÉFENSE RÉUSSIE : Kyverno a bloqué l'image non signée ! 🛡️

[Attaque 3] Tentative de déploiement depuis un registre non autorisé...
allowed-registries:
  verifier-registry: 'validation error: Image refusée : seules les images
  de ghcr.io/khalidouatik/ sont autorisées.'
--> DÉFENSE RÉUSSIE : Kyverno a rejeté le registre externe ! 🛡️

[Attaque 4] Tentative de déploiement d'une image MODIFIÉE APRÈS SIGNATURE...
resource Pod/app/pirate-tampered was blocked due to the following policies
verify-image-signature:
  verifier-signature-cosign: 'failed to verify image ...:1.0.0-tampered:
  .attestors[0].entries[0].keys: no signatures found'
--> DÉFENSE RÉUSSIE : digest ≠ signature, Kyverno a bloqué l'image altérée ! 🛡️
```

Autres preuves versionnées dans `livrables/captures/` : pods Running (`pods-running.txt`), politiques en Enforce (`politiques-enforce.txt`), vérification de la signature (`cosign-verify.txt`). Le registry GHCR étant **public**, chaque preuve est rejouable directement : `cosign verify --key cosign.pub ...` fonctionne sans authentification.

6. Positionnement SLSA & limites (honnête)

Exigence	Atteint	Justification
L1 — provenance existe	✅	Attestation <code>slsaprovenance</code> générée, attachée, vérifiée par Kyverno à l'admission.
L2 — build hébergé + provenance signée par le service de build	✅ via la CI uniquement	Le workflow GitHub Actions signe en keyless : l'identité du builder est celle du workflow (OIDC), journalisée dans Rekor.
L3 — build isolé, provenance infalsifiable	✗	Exigerait un builder durci type <code>slsa-github-generator</code> .

Limites assumées :

- La **voie locale** de la démo reste de niveau **L1** : le `provenance.json` y est écrit à la main (déclaratif). Seule la voie CI atteint L2. Nous l'annonçons tel quel.
- Les politiques Kyverno vérifient la **clé locale** (variante A). Les images signées en keyless par la CI nécessitent la **variante B** (bloc `keyless` fourni en commentaire dans `03-verify-signature.yaml`, avec

l'identité exacte du workflow). En production, on ne garderait que le keyless : aucune clé privée à protéger.

- Le SBOM inventorie, il n'**audite** pas : une backdoor de type XZ Utils dans une dépendance « légitime » ne serait pas détectée par cette chaîne.
- Voir le threat model ([livrables/Threat-Model.md](#)) pour la couverture détaillée menace par menace.

7. Reproductibilité

```
# 1. Cluster local
kind create cluster --name scs --config cluster/kind-config.yaml

# 2. Kyverno 1.12.6
kubectl create -f https://github.com/kyverno/kyverno/releases/download/v1.12.6/install.yaml

# 3. Namespace (+ secrets registry – OPTIONNELS : le package GHCR est public.
#    Requis uniquement si vous reproduisez avec votre propre registry privé.)
kubectl create namespace app
kubectl create secret docker-registry ghcr-secret -n app \
  --docker-server=ghcr.io --docker-username=<user> --docker-password=<PAT>
kubectl create secret docker-registry kyverno-registry-credentials -n kyverno \
  --docker-server=ghcr.io --docker-username=<user> --docker-password=<PAT>

# 4. Politiques (ou ./setup-cluster.sh pour une installation personnalisée automatique)
kubectl apply -f policies/kyverno/

# 5. Démo attaque/défense (attack-tamper.sh prépare l'image piégée du scénario 4)
./attack-tamper.sh
./demo.sh
```

⚠ **Versions requises** : Kyverno 1.12.x + cosign **v2.x** pour la signature (cf. incident de format documenté en §3) — ou Kyverno récent compatible avec le format bundle de cosign v3.

Threat model — Chaîne d'approvisionnement logicielle

- **Groupe** : Khalid OUATIK · Omar Mohammed HASSAS · Loic STEVE · Hadama TOURE
- **Fork** : <https://github.com/KhalidOuatik/projet-technique>
- **Date** : 08 juillet 2026

Objectif : montrer le raisonnement **menaces** → **contrôles** → **couverture**, en s'appuyant sur les contrôles réellement déployés dans ce POC (et pas seulement « on a installé des outils »).

1. Actif à protéger

L'artefact final — l'image conteneur `ghcr.io/khalidouatik/scs-demo-app` — qui tourne en production doit être **exactement** celui produit à partir du code revu, par notre chaîne de build, **sans altération** entre le build et le déploiement.

Propriétés de sécurité visées :

Propriété	Signification concrète dans ce POC
Intégrité	Le contenu de l'image déployée = le contenu de l'image buildée (garanti par le digest SHA-256).
Authenticité	L'image est signée par notre clé cosign (ou notre identité OIDC en CI).
Traçabilité	Une attestation de provenance SLSA décrit qui/quoi/d'où/quand pour chaque image.
Transparence	Le SBOM inventorie tous les composants → réponse en secondes à « suis-je affecté par la CVE du jour ? ».

2. Surface d'attaque & acteurs de menace

La chaîne complète, avec les points d'attaque possibles :



Acteurs de menace considérés :

- **Attaquant amont** : compromet une dépendance open source (cas XZ Utils 2024).
- **Attaquant CI** : compromet le runner ou une étape du pipeline (cas SolarWinds 2020, Codecov 2021).
- **Attaquant registry** : substitue ou modifie une image dans le registry, ou publie un typosquat (dependency confusion 2021).
- **Insider / accès cluster volé** : déploie directement une image pirate avec un kubeconfig compromis.
- **Développeur négligent** : déploie un tag mutable (`:latest`), une image non signée ou non scannée.

3. Table menaces → contrôles → couverture

Chaque ligne correspond à un contrôle **effectivement déployé et testé** dans ce POC (voir le rapport, section démonstration attaque/défense).

#	Menace	Vecteur	Contrôle mis en place	Preuve dans le POC	Couverture	Risque résiduel
T1	Artefact altéré après signature	tag mutation / substitution dans le registry	Signature cosign liée au digest + politique Kyverno <code>verify-image-signature</code> (Enforce)	Attaque 4 de <code>demo.sh</code> : l'image modifiée est refusée (<code>signature mismatch</code>)	Forte	Le build lui-même reste de confiance implicite
T2	Déploiement non autorisé (image inconnue)	accès cluster (kubeconfig volé, insider)	Admission Kyverno en mode Enforce : signature requisite pour <code>scs-demo-app*</code>	Attaque 1 de <code>demo.sh</code> : image non signée refusée (<code>no matching signatures</code>)	Forte	RBAC/audit du cluster à durcir (hors périmètre) ; les images hors du pattern <code>scs-demo-app*</code> ne sont contraintes que par la politique registry
T3	Dépendance vulnérable ou piégée	amont (PyPI, image de base)	SBOM Syft (SPDX + CycloneDX) + gate Grype <code>fail-on: critical, only-fixed: true</code> (local et CI)	Test local sur image <code>Flask==2.0.1</code> : exit code 2, pipeline stoppé	Moyenne	0-day, CVE sans correctif, backdoor non répertoriée (type XZ : un SBOM ne détecte pas un code malveillant « légitime »)
T4	Origine inconnue / build non tracé	image buildée hors chaîne officielle	Attestation de provenance (type <code>slsaprovenance</code>) exigée par la politique <code>require-provenance-attestation</code>	<code>cosign tree</code> montre l'attestation attachée ; Kyverno la vérifie à l'admission	Forte (présence + signature)	Le contenu de la provenance locale est déclaratif : falsifiable tant que le build n'est pas isolé (SLSA L3)
T5	Substitution silencieuse par tag mutable	<code>:latest</code> repointé vers une image piégée	Politique <code>disallow-latest-tag</code> + déploiement par digest dans <code>k8s/deployment.yaml</code>	Attaque 2 de <code>demo.sh</code> : <code>:latest</code> refusé	Forte	—

#	Menace	Vecteur	Contrôle mis en place	Preuve dans le POC	Couverture	Risque résiduel
T6	Registry pirate / typosquat	image tirée d'un registry externe	Politique <code>allowed-registries</code> : seul <code>ghcr.io/khalidouatik/</code> est autorisé	Attaque 3 de <code>demo.sh</code> : <code>nginx:alpine</code> (Docker Hub) refusé	Forte	Compromission du compte GHCR lui-même (mitigée par la signature : T1)
T7	Compromission du pipeline CI	secret volé, étape modifiée (Codecov)	Signature keyless en CI : identité OIDC du workflow, journalisée dans Rekor ; aucune clé longue durée stockée dans la CI	Workflow <code>supply-chain.yml</code> : <code>cosign sign keyless, log public Rekor</code>	Moyenne	Un attaquant contrôlant le repo (droits push sur <code>main</code>) peut modifier le workflow lui-même → protections de branche + revue requises (hors périmètre)

4. Défense en profondeur : quel contrôle rattrape quel échec

Le point clé de l'architecture : **aucun contrôle n'est seul**. Si un maillon casse, le suivant bloque.

- Le scan Gype laisse passer une CVE ? → l'image reste signée et tracée : le SBOM attesté permet de savoir **immédiatement** quelles images sont affectées quand la CVE est publiée.
- Le registry est compromis et l'image remplacée ? → le digest change → la signature ne correspond plus → **Kyverno refuse** (démonstré : attaque 4).
- Quelqu'un contourne la CI et pousse une image à la main ? → pas de signature avec notre clé → **refusée** (démonstré : attaque 1).
- Quelqu'un a un accès cluster ? → l'admission controller s'applique **avant** la création du Pod, quel que soit l'utilisateur.

5. Hors périmètre / non couvert (assumé)

- Compromission du build lui-même** (runner CI ou poste local) : exigerait SLSA L3 (build hermétique, isolé, provenance infalsifiable) — hors périmètre de ce POC.
- Sécurité du poste développeur** et des secrets en amont (PAT GitHub, clé cosign locale).
- Vulnérabilités 0-day** ou sans correctif (`only-fixed: true` les exclut délibérément du gate pour rester actionnable).
- Code malveillant « légitime »** dans une dépendance (type XZ) : le SBOM inventorie, il n'audite pas.
- RBAC / audit / network policies** du cluster : le POC se concentre sur l'admission des images.

6. Niveau SLSA visé vs atteint (honnête)

Exigence	Visé	Atteint	Justification
L1 — provenance existe	✓	✓	Attestation de provenance générée et attachée à l'image (<code>cosign attest --type slsaprovenance</code>), vérifiée par Kyverno.
L2 — build hébergé + provenance signée par le service de build	✓	✓ via la CI uniquement	Le workflow GitHub Actions builde, signe (keyless OIDC) et atteste : l'identité du builder est celle du workflow, journalisée dans Rekor. ⚠ La voie locale de la démo (clé cosign + <code>provenance.json</code> écrit à la main) reste L1 : la provenance y est déclarative.
L3 — build isolé, provenance infalsifiable	✗	✗	Exigerait un builder durci (ex. <code>slsa-github-generator</code> en job séparé) empêchant le job de build de manipuler la provenance. Identifié comme prochaine étape.

Limite assumée : les politiques Kyverno du cluster vérifient la **clé locale** (variante A). Les images signées en keyless par la CI nécessitent la variante B (bloc `keyless` fourni en commentaire dans `03-verify-signature.yaml`). En production, on ne garderait **que** la variante keyless : plus de clé privée à protéger, identité du builder vérifiable publiquement.

« On ne fait pas confiance, on vérifie. »